UNIVERSIDAD MARIANO GÁLVEZ DE GUATEMALA

FACULTAD DE INGENIERÍA EN SISTEMAS DE INFORMACIÓN

PROGRAMA DE POSGRADOS

TAREA SEMANA #2

Autor (es):

Mancia García Walter Ronaldo wmarciag@miumg.edu.gt

Profesor

CESAR GIOVANNI RODRIGUEZ MINAS

Curso

Seguridad en Aplicaciones– Cuarto Trimestre – 2026

Guatemala, 2026

Informe de Modelo de Amenazas con OWASP Threat Dragon

Aplicación evaluada: E-Commerce Platform (React + Node.js/Express + MySQL + Stripe + Cloudinary)

Curso: Seguridad en Aplicaciones, Maestría en Seguridad Informática, Universidad Mariano Gálvez de Guatemala

Autor: Walter Mancia 1793-18-7411 **Fecha:** 9 de octubre de 2026 **Versión del modelo:** 1.0

1. Alcance y levantamiento de arquitectura

Alcance evaluado: SPA web, API REST, base de datos, contenedores Docker, integración con Stripe (pagos) y Cloudinary (imágenes). No existe aplicación móvil, middleware ni integración bancaria directa; la pasarela de pago es Stripe.

Arquitectura lógica: el navegador carga la SPA servida por Nginx (puerto 80). La SPA consume la API Express (puerto 3000, prefijo `/api/v1`) con JWT en el header `Authorization`. La API usa Clean Architecture (interfaces → application → infrastructure) y persiste en MySQL 8 mediante Sequelize. Los pagos se confirman con webhooks firmados de Stripe. Todo se orquesta con `docker-compose`.

1.1 Inventario de componentes

ID	Componente	Tecnología	Exposición	Criticidad
C01	SPA	React 18, Vite, Zustand, Axios	Internet (navegador)	Alta
C02	Servidor web estático	Nginx alpine, puerto 80, sin TLS	Internet	Alta
C03	API REST	Node 20, Express 5, JWT, bcryptjs	Internet, puerto 3000	Crítica
C04	Base de datos	MySQL 8.0, Sequelize, puerto 3306 publicado	Host	Crítica
C05	Documentación API	Swagger UI en `/api/docs`	Internet	Baja
C06	Pasarela de pagos	Stripe (PaymentIntents + webhook)	Tercero	Crítica
C07	Almacén de imágenes	Cloudinary (multer-storage-cloudinary)	Tercero	Media
C08	Orquestación	Docker Compose, migraciones y seeders al arrancar	Host	Alta
C09	Dependencias npm	express, jsonwebtoken, multer, stripe, etc.	Cadena de suministro	Alta

1.2 Inventario de activos de información

ID	Activo	Clasificación	Ubicación
A01	Credenciales (hash bcrypt, cost 10)	Confidencial	MySQL `users`
A02	Access token (15 min) y refresh token (7 días)	Confidencial	localStorage del navegador
A03	Datos personales (nombre, email, rol)	Confidencial	MySQL `users`
A04	Órdenes, ítems, totales, `stripe_payment_id`	Confidencial	MySQL `orders`, `order_items`
A05	Datos de tarjeta	Restringido (nunca toca el backend si se usa Stripe Elements)	Stripe
A06	Catálogo, precios y stock	Integridad crítica	MySQL `products`

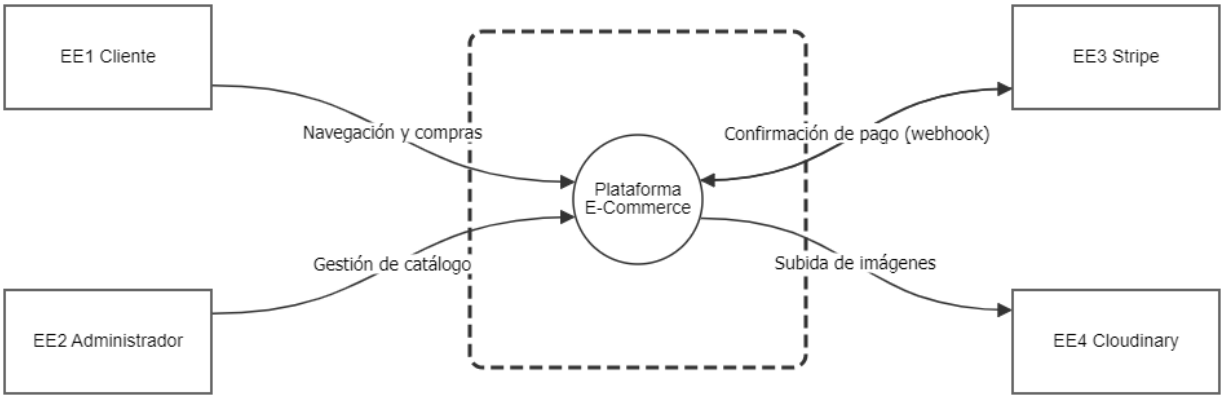
ID	Activo	Clasificación	Ubicación
A07	Secretos: `JWT_SECRET`, `JWT_REFRESH_SECRET`, `STRIPE_SECRET_KEY`, `STRIPE_WEBHOOK_SECRET`, `DB_PASSWORD`, claves Cloudinary	Secreto	Variables de entorno / `.env`
A08	Imágenes de producto	Público	Cloudinary

1.3 Clasificación de flujos por criticidad

Criticidad	Flujos
Crítica	Credenciales y tokens (F03, F04), checkout (F09), pagos y webhook (F10, F12)
Alta	Carrito (F08), administración de catálogo (F13), persistencia de usuarios (F06)
Media	Subida de imágenes (F14), tokens en localStorage (F05)
Baja	Catálogo público (F07), estáticos (F02), Swagger (F16)

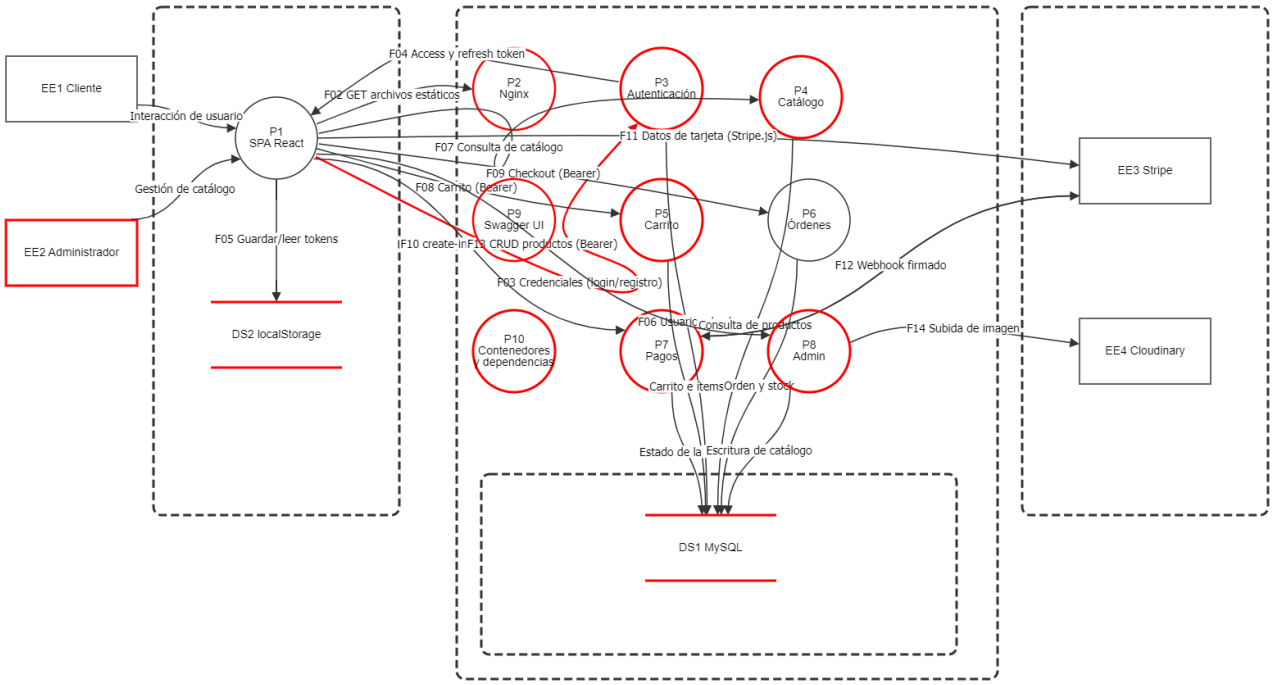
2. Diagramas de flujo de datos (DFD)

2.1 Nivel 0: contexto



Entidades externas: Cliente (EE1), Administrador (EE2), Stripe (EE3), Cloudinary (EE4). Un atacante anónimo opera desde Internet, el mismo lado que el cliente.

2.2 Nivel 1: procesos, almacenes y límites de confianza



Procesos: P1 SPA, P2 Nginx, P3 Autenticación (`/auth`), P4 Catálogo (`/products`, `/categories`), P5 Carrito (`/cart`), P6 Órdenes y checkout (`/orders`), P7 Pagos (`/payments`: intent y webhook), P8 Administración (CRUD de productos y subida de imágenes).

Almacenes: DS1 MySQL (users, products, categories, carts, cart_items, orders, order_items), DS2 localStorage.

Trust boundaries:

ID	Límite	Qué separa
TB1	Navegador ↔ servidor	Cliente no confiable y backend
TB2	Internet ↔ host Docker	Exposición de los puertos 80, 3000 y 3306
TB3	API ↔ base de datos	Capa de aplicación y datos
TB4	Plataforma ↔ SaaS externos	Stripe y Cloudinary

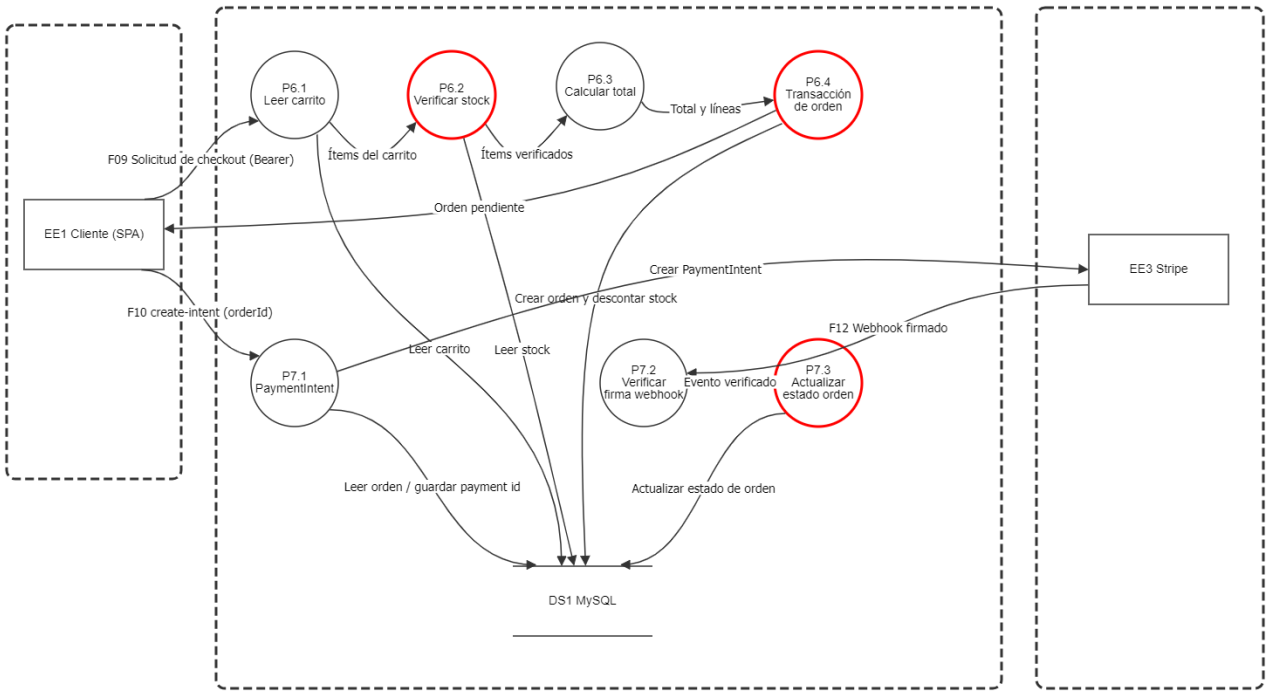
Flujos de datos:

ID	Origen → Destino	Datos	Protocolo
F02	SPA → Nginx	Archivos estáticos	HTTP
F03	SPA → P3	email y contraseña	HTTP/JSON
F04	P3 → SPA	access y refresh token	HTTP/JSON
F05	SPA ↔ DS2	Tokens	API del navegador
F06	P3 ↔ DS1	Usuario y hash	SQL (sin TLS configurado)
F07	SPA → P4 → DS1	Búsqueda y paginación	HTTP / SQL
F08	SPA → P5 → DS1	Producto y cantidad	HTTP + Bearer / SQL
F09	SPA → P6 → DS1	Solicitud de checkout, orden y stock	HTTP + Bearer / SQL
F10	SPA → P7 ↔ Stripe	orderId, clientSecret	HTTP / HTTPS

ID	Origen → Destino	Datos	Protocolo
F11	SPA → Stripe	Datos de tarjeta (Stripe.js)	HTTPS
F12	Stripe → P7 → DS1	Evento firmado, estado de orden	HTTPS / SQL
F13	Admin → P8 → DS1	Producto, precio, stock	HTTP + Bearer / SQL
F14	P8 → Cloudinary	Imagen	HTTPS
F16	Cualquiera → Swagger	Especificación de la API	HTTP

2.3 Nivel 2: checkout y pago (componente crítico)

Subproceso	Descripción	Observación de seguridad
P6.1	Leer carrito del usuario autenticado	`userId` proviene del JWT (correcto)
P6.2	Verificar stock por ítem	Se lee fuera de la transacción (T06)
P6.3	Calcular total con precio de BD	Precio del servidor (correcto), pero cantidades sin validar (T02)
P6.4	Transacción: crear orden, descontar stock	Sin bloqueo de filas; `clearCart` fuera de la transacción
P7.1	Crear o reutilizar PaymentIntent	Monto desde la orden; idempotencyKey por orden (correcto)
P7.2	Verificar firma del webhook	`constructEvent` con secret (correcto)
P7.3	Actualizar estado de la orden	No valida estado previo ni monto; no repone stock al cancelar (T07, T14)



Evidencias a generar en Threat Dragon: archivo `.json` del modelo, capturas de los DFD 0, 1 y 2, y el reporte de amenazas.

3. Análisis STRIDE

3.1 Lista de amenazas (Impacto × Probabilidad)

Escala: Crítico/Muy Alta = 4, Alto/Alta = 3, Medio/Media = 2, Bajo/Baja = 1. Puntaje = I × P. Niveles: **≥9 Crítico, 5-8 Alto, 3-4 Medio, ≤2 Bajo**. Esta regla reproduce la tabla del enunciado (Alto-Alto = Crítico, Alto-Medio = Alto, Medio-Medio = Medio, Bajo-Bajo = Bajo).

ID	Elemento	STRIDE	Amenaza	Evidencia	I	P	Pts	Nivel
T01	P3 / F03	S, D	Sin rate limiting ni bloqueo en login, registro y refresh: fuerza bruta, credential stuffing y saturación	`app.js`, `auth.routes.js`	Alto	Muy Alta	12	Crítico
T02	P5, P6	T	`quantity` sin validar: valores negativos o no enteros permiten reducir el total y aumentar stock artificialmente	`cart.service.js`, `order.service.js`, `CartItem.js`	Alto	Alta	9	Crítico
T05	F02, F03-F13	I, T	Nginx y API sin TLS: tokens y credenciales viajan en claro	`nginx.conf`, `docker-compose.yml`	Alto	Alta	9	Crítico
T23	P3, P8	S, E	Cuenta admin sin MFA ni políticas de sesión; su compromiso da control del catálogo	`auth.service.js`, `product.routes.js`	Crítico	Media	8	Alto
T12	DS1 / TB3	E, I, T	MySQL publicado en 3306 y uso de `root` como usuario de aplicación (README)	`docker-compose.yml`, README	Crítico	Media	8	Alto
T03	P1 / DS2	I, S	Access y refresh token en localStorage, sin CSP: un XSS los exfiltra	`api.js`, `authStore.js`, `nginx.conf`	Alto	Media	6	Alto
T04	P3	S, E, R	Refresh token sin rotación ni revocación, sin logout en servidor, y `refreshTokenService` no valida `is_active`	`auth.service.js`, `jwt.utils.js`	Alto	Media	6	Alto
T06	P6	T	Condición de carrera en checkout: verificación y descuento de stock sin bloqueo, riesgo de sobreventa	`order.service.js`	Medio	Alta	6	Alto
T07	P6, P7	D	Órdenes `pending` descuentan stock sin expiración, y `payment_failed` cancela sin reponer: vaciado de inventario	`order.service.js`, `payment.service.js`	Medio	Alta	6	Alto
T11	Todos	R	Sin logging de seguridad ni auditoría (solo `console.log`); acciones admin y cambios de estado no trazables	`payment.service.js`, `server.js`	Medio	Alta	6	Alto
T18	P4 / F07	D	`limit` sin tope y búsqueda `LIKE '%x%'` sin índice	`product.service.js`, `product.repository.js`	Medio	Alta	6	Alto
T19	P3	S, D	Sin validación de entrada en register/login: sin política de contraseña, longitud máxima ni formato; bcrypt con contraseñas enormes consume CPU	`auth.routes.js`, `auth.controller.js`	Medio	Alta	6	Alto
T22	C09	T	Cadena de suministro: sin SCA, sin pruebas ni CI (`"test": "echo Error..."`), versiones con ^	`package.json`	Alto	Media	6	Alto
T08	P3-P8	S, I	`cors()` sin restricción de orígenes	`app.js`	Medio	Media	4	Medio
T09	C05	I	Swagger accesible sin autenticación en producción	`app.js`	Bajo	Muy Alta	4	Medio
T10	Todos	I	El error handler devuelve `err.message` en producción (errores de Sequelize, Stripe)	`error.middleware.js`, `payment.service.js`	Medio	Media	4	Medio
T15	P3	I	Enumeración de usuarios: el registro responde 409 "correo ya registrado"	`auth.service.js`	Bajo	Muy Alta	4	Medio

ID	Elemento	STRIDE	Amenaza	Evidencia	I	P	Pts	Nivel
T16	P4	I	`GET /products/id/:id` devuelve productos inactivos (eliminados lógicamente)	`product.routes.js`, `product.repository.js`	Bajo	Alta	3	Medio
T13	C08	E	Contenedores como root, secretos en variables de entorno, migraciones y seeders en cada arranque	`Dockerfile`, `docker-compose.yml`	Alto	Baja	3	Medio
T14	P7 / F12	T, D	Webhook sin idempotencia por `event.id`, sin validar estado previo ni monto, endpoint público sin límites	`payment.service.js`, `app.js`	Alto	Baja	3	Medio
T20	P3	E	JWT sin fijar algoritmo, `iss` ni `aud`; el rol del token no se revalida contra la BD durante 15 min	`jwt.utils.js`, `auth.middleware.js`	Alto	Baja	3	Medio
T17	P8	T, E	Mass assignment: `...productData` y `update(productData)` aceptan campos arbitrarios (slug, is_active, image_url)	`product.service.js`	Medio	Baja	2	Bajo
T21	P8 / F14	T	Subida de imágenes validada por tipo declarado; depende de Cloudinary y de que el admin no esté comprometido	`cloudinary.utils.js`	Medio	Baja	2	Bajo

3.2 Matriz STRIDE consolidada

Elemento	S	T	R	I	D	E
EE1 Cliente / EE2 Admin	T01, T23		T11			
P1 SPA / DS2 localStorage	T03			T03		
P2 Nginx		T05		T05		
P3 Autenticación	T01, T04, T19		T04	T15	T01, T19	T04, T20
P4 Catálogo				T16	T18	
P5 Carrito		T02				
P6 Checkout		T02, T06			T07	
P7 Pagos / F12		T14	T11	T10	T14	
P8 Administración	T23	T17, T21	T11			T23
DS1 MySQL		T12		T12		T12
Flujos F02-F13	T08	T05		T05, T09, T10		
Dependencias / C08		T22				T13

3.3 Registro de validación técnica

Fecha	Participante y rol	Amenazas revisadas	Resultado
[fecha]	[nombre], Desarrollo	T01-T23	[Confirmadas / ajustes]
[fecha]	[nombre], Arquitectura	T01-T23	[Confirmadas / ajustes]

Debe completarse con una revisión real (docente, compañero o desarrollador). Si el proyecto es individual, documenta una autorrevisión y una revisión cruzada.

4. Priorización de riesgos

4.1 Matriz de riesgos (cantidad de amenazas por celda)

Impacto \ Probabilidad	Baja	Media	Alta	Muy Alta
Crítico		T12, T23 (Alto)		
Alto	T13, T14, T20 (Medio)	T03, T04, T22 (Alto)	T02, T05 (Crítico)	T01 (Crítico)
Medio	T17, T21 (Bajo)	T08, T10 (Medio)	T06, T07, T11, T18, T19 (Alto)	
Bajo			T16 (Medio)	T09, T15 (Medio)

4.2 Top 10 de hallazgos prioritarios

Ordenados por puntaje; los empates se resuelven por criticidad del activo afectado.

#	ID	Hallazgo	Nivel
1	T01	Sin rate limiting en autenticación	Crítico
2	T02	Cantidades negativas o sin validar en carrito y checkout	Crítico
3	T05	Tráfico sin TLS	Crítico
4	T23	Admin sin MFA	Alto
5	T12	MySQL expuesto y usuario root	Alto
6	T04	Refresh token sin rotación ni revocación	Alto
7	T03	Tokens en localStorage sin CSP	Alto
8	T06	Condición de carrera en stock	Alto
9	T07	Stock retenido por órdenes sin pagar	Alto
10	T19	Sin validación de entrada en autenticación	Alto

5. Catálogo de casos de abuso (amenazas críticas y altas)

ID	Amenaza	Actor y objetivo	Vector y precondiciones	Método de explotación	Impacto	Control preventivo	Control detectivo	Acción correctiva
AC-01	T01	Atacante externo: acceso no autorizado a cuentas	`/auth/login` público; listas de credenciales filtradas	Intentos masivos automatizados sin límite	Toma de cuentas, saturación de CPU (bcrypt)	Rate limiting por IP y cuenta, bloqueo progresivo, MFA, captcha	Alertas por picos de 401, métrica de fallos por IP	Bloquear IP, forzar reset de contraseñas afectadas
AC-02	T02	Usuario registrado malicioso: pagar menos o inflar stock	Cuenta propia (registro abierto)	`POST /cart/items` con `quantity: -N` junto a un artículo caro; checkout resta cantidad negativa	Pérdida económica, inventario corrupto	Validar `quantity` entero ≥ 1 y tope; CHECK en BD; recalcular y validar total > 0	Alertas por órdenes con total anómalo; reconciliación de stock	Revertir órdenes y stock, auditar cuentas
AC-03	T05	Atacante en red (Wi-Fi público): robar credenciales y tokens	Tráfico HTTP sin TLS	Sniffing o MITM	Secuestro de sesión (incluso admin)	TLS 1.2+ en Nginx y API, HSTS, redirección 80→443	Monitoreo de accesos HTTP en claro	Revocar tokens, rotar secretos
AC-04	T23	Atacante externo: control del catálogo	Credenciales admin reutilizadas o robadas	Login válido; cambiar precios, crear productos	Fraude de precios, defacement	MFA obligatorio, contraseñas robustas, sesiones cortas para admin	Log de auditoría de cambios admin, alertas de cambio de precio	Revertir cambios, revocar sesiones, rotar credenciales
AC-05	T12	Atacante en la red del host: leer y modificar la BD	Puerto 3306 alcanzable, contraseña débil o `root`	Conexión directa a MySQL	Fuga total de PII y manipulación de órdenes	No publicar 3306, usuario con mínimo privilegio, red interna, TLS a BD	Auditoría de conexiones MySQL	Rotar credenciales, restaurar desde respaldo
AC-06	T04	Atacante con token robado: persistir acceso	Refresh token obtenido (XSS, malware)	Renovar access token durante 7 días, aun si el usuario fue desactivado	Acceso prolongado	Rotación con detección de reuso, almacenamiento revocable, validar `is_active`	Alertas de refresh desde IP o dispositivo nuevo	Revocar familia de tokens, forzar re-login
AC-07	T03	Atacante con XSS: robar tokens	Inyección en la SPA o en una dependencia	Script que lee `localStorage`	Suplantación de usuario	CSP estricta, tokens en cookie HttpOnly + SameSite, sanitización	Reportes CSP, monitoreo de uso anómalo	Invalidar sesiones, parchear la inyección
AC-08	T06	Usuario legítimo o bot: sobreventa	Dos checkouts simultáneos	Peticiones paralelas aprovechando verificación fuera de la transacción	Stock negativo, pedidos incumplibles	`SELECT ... FOR UPDATE` o decremento atómico condicional; CHECK `stock >= 0`	Alerta por stock < 0	Cancelar y reembolsar, reconciliar
AC-09	T07	Competidor malicioso: agotar inventario	Cuenta registrada	Crear órdenes `pending` sin pagar para reservar todo el stock	Pérdida de ventas	Expiración de órdenes pendientes y reposición, límite de órdenes pendientes por usuario	Métrica de órdenes pendientes por usuario	Job de limpieza, bloquear cuenta
AC-10	T11	Insider o atacante: negar acciones	Sin logs de auditoría	Modificar precios o estados sin rastro	Imposible atribuir o investigar	Logging estructurado con usuario, IP y acción	SIEM con alertas	Reconstruir con respaldos de BD
AC-11	T18	Atacante: degradar el servicio	Endpoints públicos	`?limit=100000` y búsquedas `%` repetidas	Agotamiento de BD y memoria	Tope de `limit` (p. ej. 50), índices, rate limiting, caché	Alertas de latencia y CPU	Aplicar límites, bloquear origen

ID	Amenaza	Actor y objetivo	Vector y precondiciones	Método de explotación	Impacto	Control preventivo	Control detectivo	Acción correctiva
AC-12	T19	Atacante: DoS o cuentas débiles	`/auth/register` público	Contraseñas gigantes o triviales; cuentas masivas	Consumo de CPU, cuentas fácilmente comprometibles	Validación con express-validator (longitud 12-72, complejidad, email), límites de body	Alertas por registros masivos	Purgar cuentas, endurecer validación
AC-13	T22	Atacante de cadena de suministro	Paquete npm comprometido	Dependencia maliciosa en `npm install`	Ejecución de código en backend y robo de secretos	`npm audit` / SCA en CI, lockfile, versiones fijas, pruebas	Dependabot, monitoreo de CVE	Pin y parcheo, rotar secretos

Riesgos residuales esperados tras mitigar: T01 y T05 pasan a Bajo; T02 a Bajo; T23 a Medio (riesgo de phishing del segundo factor); T12 a Bajo; T22 permanece Medio (dependencia inherente de terceros); T03 a Medio hasta migrar a cookies HttpOnly.

6. Controles de seguridad

6.1 Controles ya existentes (fortalezas)

bcrypt para contraseñas; mensaje genérico en login (evita enumeración en login); access y refresh con secretos distintos y vida corta (15 min); RBAC con `authorize('admin')` en rutas de escritura; órdenes y carrito filtrados por `userId` del token (previene IDOR); precios calculados en servidor; transacción en checkout; PaymentIntent con idempotencyKey; verificación de firma del webhook de Stripe; eliminación lógica de productos; DTOs que no exponen el hash; validación con express-validator en productos; `.env` excluido de Docker; datos de tarjeta delegados a Stripe (reduce alcance PCI).

6.2 Mapeo amenaza → control

Capa	Control recomendado	Amenazas	Estado
Aplicación	MFA (TOTP) obligatorio para admin y opcional para clientes	T23, T01	No implementado
Aplicación	Validación de entrada con express-validator en auth, carrito, checkout y paginación (`quantity` entero ≥ 1, `limit` ≤ 50)	T02, T18, T19	Parcial (solo productos)
Aplicación	Gestión de sesiones: rotación de refresh token, lista de revocación, logout en servidor, validar `is_active`	T04, T20	No implementado
Aplicación	Checkout con bloqueo de filas y decremento atómico; expiración de órdenes pendientes con reposición	T06, T07	No implementado
Aplicación	Lista blanca de campos al crear o actualizar productos	T17	No implementado
Aplicación	Respuesta genérica en registro; error handler sin detalles internos en producción	T15, T10	No implementado
Cifrado	TLS en Nginx y API, HSTS, TLS hacia MySQL	T05, T12	No implementado
Frontend	CSP, cabeceras de seguridad (helmet y Nginx), tokens en cookie HttpOnly + SameSite	T03	No implementado
API	Rate limiting (`express-rate-limit`), bloqueo progresivo en `/auth`, límites en webhook	T01, T14, T18	No implementado
API	CORS con lista blanca de orígenes; Swagger solo en desarrollo o protegido	T08, T09	No implementado
API	JWT con `algorithms: ['HS256']`, `iss`, `aud`; considerar OIDC con proveedor de identidad	T20	No implementado
API	Webhook idempotente por `event.id`, validación de estado y monto	T14	Parcial (firma)
Infraestructura	No publicar 3306; usuario de BD con mínimo privilegio; red Docker interna	T12	No implementado
Infraestructura	Contenedores no root (`USER node`), imágenes escaneadas, gestor de secretos (Docker secrets o Vault), migraciones fuera del arranque	T13	No implementado
Monitoreo	Logging estructurado centralizado (pino/winston), auditoría de acciones admin, SIEM y alertas (picos de 401, cambios de precio, stock < 0)	T11, T01, T04	No implementado
Cadena de suministro	`npm audit` y SCA en CI, lockfile, Dependabot, pruebas automatizadas, SAST	T22	No implementado

7. Matriz de riesgos y remediación

Riesgo	Severidad	Responsable (propuesto)	Acción correctiva	Fecha objetivo
T01	Crítico	Backend	Rate limiting y bloqueo progresivo en `/auth/*`	2026-10-30

Riesgo	Severidad	Responsable (propuesto)	Acción correctiva	Fecha objetivo
T02	Crítico	Backend	Validar `quantity` (entero ≥ 1, tope) y total > 0; CHECK en BD	2026-10-23
T05	Crítico	DevOps	TLS en Nginx y API, HSTS, redirección a HTTPS	2026-11-15
T23	Alto	Backend	MFA TOTP para admin	2026-12-15
T12	Alto	DevOps	Quitar puerto 3306, usuario de BD dedicado, red interna	2026-11-06
T04	Alto	Backend	Rotación y revocación de refresh token, validar `is_active`	2026-12-15
T03	Alto	Frontend/Backend	CSP y migración a cookies HttpOnly	2027-01-31
T06	Alto	Backend	Bloqueo de filas y decremento atómico de stock	2026-11-20
T07	Alto	Backend	Expiración de órdenes pendientes y reposición	2026-12-15
T11	Alto	DevOps/Backend	Logging estructurado y auditoría	2027-01-31
T18	Alto	Backend	Tope de paginación e índice de búsqueda	2026-11-06
T19	Alto	Backend	Validadores de auth y política de contraseñas	2026-11-06
T22	Alto	DevOps	SCA/SAST y pruebas en CI	2027-01-31
T08, T09, T10, T15, T16	Medio	Backend	CORS restringido, Swagger protegido, errores genéricos, filtrar inactivos	2026-12-31
T13, T14, T20	Medio	DevOps/Backend	Contenedores no root, webhook idempotente, JWT endurecido	2027-03-31
T17, T21	Bajo	Backend	Lista blanca de campos, validación de imágenes	2027-06-30

8. Roadmap de mejoras

Horizonte	Hallazgos	Foco
Corto plazo (0-3 meses)	T01, T02, T05, T12, T18, T19, T06	Cerrar vectores críticos: validación de entrada, rate limiting, TLS, exposición de BD y consistencia de stock
Mediano plazo (3-6 meses)	T23, T04, T07, T03, T11, T22, T08, T09, T10, T15, T16	Sesiones y MFA, expiración de órdenes, CSP y cookies HttpOnly, logging, CI con SCA/SAST
Largo plazo (6-12 meses)	T13, T14, T20, T17, T21	Hardening de contenedores y secretos, SIEM, OIDC, pruebas de penetración y revisión periódica del modelo en cada release

Mantenimiento del modelo: el archivo `.json` de Threat Dragon se versiona en Git junto al código y se revisa en cada cambio de arquitectura o antes de cada release (SSDLC).